

1. Design the following functions:

int mp(int n) – to return multiplicative Persistence of n

The **Multiplicative Persistence** of a number is the number of steps required to reduce it to a single-digit number by **repeatedly multiplying its digits**.

int sum(int a, int b) returns sum of **Multiplicative Persistence** of each prime number between a and b using the function mp()

Write the driver code (main function to execute the program)

2. Design the following functions:

int dr(int n) to return digital root of n

The **Digital Root** of a number is obtained by repeatedly summing the digits until only a **single-digit** number remains.

int lcmOfDr(int a, int b) to return lcm of Digital Roots of a and b

3. Design the following functions:

boolean isCoprime(int n, int k) – returns true if n and k are coprime.

n is coprime with k if $\text{gcd}(n, k) == 1$

int phi(int n) returns Euler's Totient of n

Euler's Totient Function, written as $\phi(n)$, counts the number of integers between **1 and n** that are **coprime with n**.

$\phi(n) = |\{i \mid 1 \leq i \leq n, \text{gcd}(i, n) == 1\}|$

boolean isPrimitiveRoot(int n, int g) returns true if g is primitive root modulo n

Check if g is a Primitive Root modulo n

Compute $\phi(n)$ (Euler's Totient Function)

Find all prime factors of $\phi(n)$.

For each prime factor p, check-

$$g^{\phi(n)/p} \bmod n \neq 1$$

If this holds for all prime factors, then g is a primitive root modulo n.

4. Design the following functions:

boolean isPerfectSquare(int n) returns true if n is perfect square

boolean isInFibonacci(int n) checks if n is in Fibonacci sequence or not

n belongs to Fibonacci sequence $\Leftrightarrow (5n^2+4)$ or $(5n^2-4)$ is a perfect square.

5. Design the following functions:

int sumOfParts(int n) take a number and find sum of last one, last two, last three up to last n digits

Example : n= 12345

$$s = 5 + 45 + 345 + 2345 + 12345 \quad \text{or } s = \sum_{i=1}^n n \% 10^i$$

6. overload a function patterns() as follows :

patterns()

```
* * * * *
* * * * 1
* * * 2 1
* * 3 2 1
* 4 3 2 1
```

patterns(int n)

$$S = 1 + \frac{1+2}{1*2} + \frac{1+2+3}{1*2*3} + \frac{1+2+3+4}{1*2*3*4} + \dots + \frac{1+2+3+4+5\dots n}{1*2*3*4*5\dots n}$$

patterns(long n)

$$\sum_{i=1}^n \frac{\sum_{j=1}^i j}{\prod_{j=1}^i j}$$

```
      1
     1 2
    1 2 3
   1 2 3 4
  1 2 3 4 5
 1 2 3 4
 1 2 3
 1 2
 1
```

7. Design the following functions

String reverse(String s) to return reverse of the parameterized String

boolean isPalindrome(String s) to check if s is a palindrome or not using the function reverse

In the main function, take a sentence and print only palindrome words from it using the functions defined above.

8. Design the following functions

void sort(char a[]) to sort the character array without using any standard sorting algorithm

String sorted(String s) to convert the String s to a character array and sort it using the function sort() and return the sorted array.

Write the driver code